Multi-Agent DRL-Based Large-Scale Heterogeneous Task Offloading for Dynamic IoT Systems

Xiao He, Shanchen Pang, Haiyuan Gui, Kuijie Zhang, Nuanlai Wang, *Graduate Student Member, IEEE*, and Xue Zhai

***Abstract*—In dynamic IoT system, the device may generate multiple heterogeneous computational tasks, that require CPU and GPU co-processing, in each period. Furthermore, different heterogeneous computing tasks have specific requirements for GPU resource types. Realizing real-time scheduling and processing of large-scale hybrid computing tasks with high heterogeneity and dense quantity has become an urgent problem. First, we propose a cloud-based task processing framework that uses multi-level feedback queues to ensure the fairness of large-scale task parallel computing. Second, we decoupled the original problem into a series of mixed-integer nonlinear programming problems using Lyapunov optimization, aiming to reduce the solution complexity of the real-time scheduling problem. Finally, we propose a multi-agent reinforcement learning algorithm, employing long and short-term memory networks with parameter resetting, to generate task offloading decisions in near real-time based on partially knowable future information. Through extensive simulation experiments, we have demonstrated that our algorithm can reduce the average task processing time by approximately 19.95% and enhance the task processing capability of the IoT system by roughly 12.43%, especially in large-scale hybrid computing task systems.**

***Index Terms*—Internet of Things, heterogeneous task, reinforcement learning, Lyapunov optimization, multi-level feedback queue.**

I. Introduction

With the rapid development of the Internet of Things (IoT) and 5G technologies, each end device (ED) is evolving towards cloudification [1], [2]. As a large-scale data and processing center, cloud computing provides rich computing resources for EDs, but it is difficult to respond to task requests quickly and efficiently. Mobile Edge Computing (MEC) has emerged as a distributed computing paradigm to address the issue of prolonged response times. MEC processes part or all of the tasks from EDs by offloading them to nearby Mobile Edge Server (MES) nodes, significantly reducing network transmission latency [3], [4].

Received 29 June 2024; revised 19 November 2024; accepted 20 December 2024. Date of publication 24 December 2024; date of current version 24 February 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFA1000102 and Grant 2021YFA1000103, and in part by the National Key Laboratory of Large-Scale Personalized Customization Systems and Technology, under Grant H&C-MPC-2023-02-03. Recommended for acceptance by Dr. Xiaoli Chu. *(Corresponding author: Shanchen Pang.)*

The authors are with the Qingdao institute of software, College of computer science and technology, China University of Petroleum, Qingdao 266580, China (e-mail: pangsc@upc.edu.cn).

Digital Object Identifier 10.1109/TNSE.2024.3521885

In IoT systems, there has been a significant increase in hybrid computing tasks that involve the collaborative processing of CPUs and GPUs, such as real-time data analytics, production process optimization, and anomaly detection. However, MES's limited computational and storage resources constrain their ability to handle large-scale task demands effectively. To address this limit, the industry has explored various approaches, such as task offloading [4], [5] and resource utilization [6], to manage tasks that are intensive in quantity and resource demands. Current works have predominantly focused on CPU-centric task processing.

GPUs leverage their powerful floating-point computation capabilities to reduce training times for artificial intelligence tasks significantly [7], thus enabling concurrent processing of multiple tasks. As a result, scheduling tasks in heterogeneous CPU-GPU resource networks has become a trend [8]. For instance, work [9] employs multiplexing techniques to enhance the resource utilization of CPUs and GPUs. Research [10] explores the computational offloading process in CPU-GPU heterogeneous resource networks in IoT environments. Despite these advances, two main challenges persist in efficiently handling large-scale hybrid computing tasks:

1) *Hybrid resource requirements for heterogeneous tasks:* The work [11] shows that tasks have specific constraints on the choice of GPU type, and some models are specifically optimized for specific generations of GPUs. As a result, these tasks require co-processing of the CPU and specific GPUs, which significantly increases the complexity of task scheduling and resource management in IoT systems.
2) *Fluctuations and large-scale of task quantity:* In dynamic IoT systems, ED may generate multiple heterogeneous tasks within each period, a significant accumulation of tasks within the IoT system and greatly increasing the complexity of large-scale task processing. Furthermore, the continuous addition of new tasks and the completion of old tasks leads to constant fluctuations in the system's task queue, further exacerbating the complexity.

These challenges raise higher demands for system task scheduling and resource management. On the one hand, IoT systems must rapidly respond to dynamic changes in IoT system states. Literature [12], [13] introduces an online offloading decision-making method based on Lyapunov optimization for fast response to tasks within a dynamic IoT system. However, current research primarily focuses on the processing of homogeneous tasks, neglecting the diversity of task types and the heterogeneous resource requirements found in real-world environment [14]. Therefore, there is an urgent need to develop

an algorithm that facilitates real-time offloading decisions for large-scale heterogeneous computing tasks with specific GPU resource demands.

On the other hand, the system must precisely manage the resource requirements of tasks. Research shows that virtualizing MES computational resources facilitates the dynamic allocation of multidimensional resources such as CPU and memory [15], [16]. However, MES struggles to allocate and manage these resources accurately due to limitations in operating system scheduling and a limited number of CPU and GPU cores [17], [18]. The industry has implemented queuing theory to improve MES's parallel processing capabilities [19], [20]. This method, however, can lead to prolonged resource monopolization by certain tasks, which reduces system processing performance. Consequently, a cloud-based online framework is urgently needed to ensure fair management of large-scale tasks.

In summary, although the industry has made progress in the real-time processing of large-scale tasks, existing scheduling methods make it difficult to meet the demands for low latency and high-quality processing of large-scale heterogeneous computing tasks. To address this, we propose an online processing framework based on the Multi-Level Feedback Queue (MLFQ). Furthermore, we propose a Cloud-based task online processing framework based on multi-level feedback queue (MLFQ). Finally, we propose a **L**ong short-term memory (LSTM) and parameter **R**eset-based **M**ulti-**A**gent reinforcement learning (LRMA) algorithm for real-time offloading decisions for newly generated tasks in each time frame. The main contributions are as follows:

- *Hybrid computing task online processing:* We model the online processing of large-scale hybrid computing tasks as a multi-stage mixed integer nonlinear programming (MINLP) problem. By employing Lyapunov optimization, we decompose this multi-stage MINLP into a series of deterministic MINLP problems. In particular, this approach allows us to provide an optimal processing solution for each task in the current timeframe when the arrival state of future tasks cannot be predicted.
- *Multi-level heterogeneous feedback queuing:* First, we build a multi-dimensional resource processing queue with joint CPU and heterogeneous CPUs to meet the heterogeneous resource requirements of hybrid computing tasks. Second, we introduce the multi-level feedback rotation scheduling to solve the problem that some tasks monopolize computational resources for a long time while others cannot be executed in time, which realizes the parallel processing of large-scale tasks.
- *Real-time decision-making algorithm design:* We propose the LRMA algorithm to realize the real-time offloading decision for each hybrid task under unpredictable conditions of future IoT system states. The algorithm introduces a parameter resetting technique to inhibit the overfitting risk due to the primacy bias [21]. It combines with the LSTM algorithm to predict the future EU state, improving the LRMA algorithm's decision feasibility.

The rest of the work is organized as follows. Section II provides related work. Section III describes the processing flow and task queuing framework for hybrid computing tasks. In Section IV, the problem is decoupled using Lyapunov optimization. In Section V, the LRMA algorithm is designed. Simulation results and conclusions are given in Sections VI and VII, respectively.

II. Related Work

A. Hybrid Computing Task Processing

Past research has focused on processing optimization for general-type CPU tasks. For example, Gao et al. [4] proposed a task online offloading algorithm for offloading and computing large-scale heterogeneous CPU tasks. In addition, literature [6], [22] explored how to reasonably allocate computational resources to realize the problem of parallel processing of heterogeneous CPU tasks. However, in real-world environments, terminals and base station (BS) devices are equipped with CPUs and other processing units such as GPUs. To fully utilize these diverse resources, researchers have begun exploring the CPU-GPU collaborative computing field. Gong et al. [8] proposed a CPU-GPU heterogeneous edge cluster, which utilizes GPUs' parallel computing capability to efficiently process large-scale tasks. Tang et al. [23] proposed a task dynamic scheduling algorithm based on a multicore CPU-GPU processor to realize real-time processing of large-scale data streams.

In addition to utilizing the parallel processing capability of GPU units, many scholars have studied the task processing problem for multi-resource type requirements such as GPUs, storage, etc [24], [25]. However, the above works focus on a single aspect of task attributes, ignoring the fact that the processing of hybrid computing tasks such as autonomous driving, industrial robotics, and smart cameras requires interoperability between CPUs and GPUs. Inspired by the dataset of "Kubernetes Scheduler Simulator" [11] that tasks require general-purpose CPU resources and specific GPU resources for collaborative processing, we study the processing problem of large-scale hybrid computing tasks.

B. MES Task Processing Framework

Cloud and edge-side base stations have a large number of computational resources. How to efficiently utilize these computational resources to achieve parallel processing of computational tasks has become a top priority. Literature [26], [27] considers the computational resources of MES as a first-come-first-served (FCFS) queue and designs a priority determination algorithm to realize multitask processing. In addition, some work used the M/M/C queueing theory approach by dividing the number of CPU cores at the MES to realize parallel multitasking [28], [29], [30]. Bebortta et al. [28] proposed a unified stochastic computational offloading framework that divides the computational resources of the MES into several copies to provide task parallel processing. Guan et al. [29] prioritized tasks in terms of task heterogeneity and system dynamics and offloaded them to the corresponding queues of the MES for processing. However, in IoT system, the pending tasks are huge, and the task resource requirements differ. When many tasks converge in the cloud, if the fairness of the task processing cannot be guaranteed, it will probably cause problems such as long waiting times and processing timeout for some tasks. The industry utilizes the multi-level feedback control protocol [31] to achieve congestion control for massive data streams. Zhao et al. [32] combine reinforcement learning and multi-level feedback task scheduling to achieve service request scheduling. Wang et al. [33] propose a hierarchical queue-based time-slicing self-adaptive scheduling algorithm that utilizes the time slice rotation to ensure the fairness of task processing. In summary, we propose a **M**ulti-level

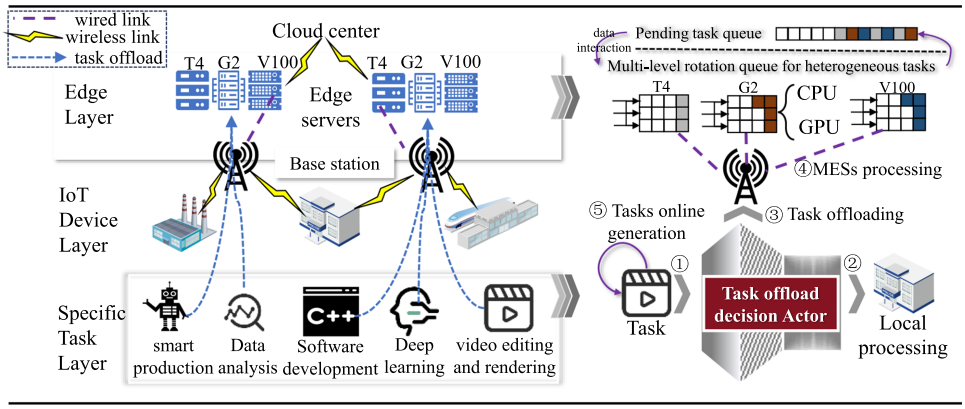

Fig. 1. Dynamic IoT system model.

Heterogeneous **F**eedback **Q**ueue (MHFQ) framework to integrate the traditional MLFQ into the heterogeneous GPU resource allocation framework for MES to meet the requirements of MES for the timeliness and fairness of large-scale task processing.

III. System Model

Fig. 1 illustrates a dynamic IoT system's three-layer architecture, comprising the task, IoT device, and edge layers. The task layer represents heterogeneous computing tasks generated by IoT ED, characterized as independent and indivisible. These tasks vary in resource demands and processing complexity. The device layer consists of various types of IoT EDs. These devices can generate multiple tasks within each time frame. Not only must EDs handle existing pending tasks, but they must also make offloading decisions for newly generated tasks during each slot. However, constrained by limited computational resources, it is difficult for IoT devices to make complex offloading decisions. They usually determine whether a task needs to be offloaded to a neighboring MES based on the observable local environment and task state. The remote cloud is divided into two main parts: the edge layer and the cloud center. The cloud center collects and distributes MES information to each ED at the start of each time slot, playing a crucial role in global resource management. Concurrently, it devises allocation strategies for all pending offloading tasks at the beginning of each slot. The edge layer contains servers that handle various heterogeneous computing tasks originating from EDs and return the results. This three-tier structure ensures the stability and responsiveness of dynamic IoT systems in complex and heterogeneous task environments through a clear division of functions and efficient task processing workflows.

Each ED is equipped with a CPU and multiple, diverse GPU. These EDs move within the communication range of BS. The EDs communicate with the BSs via wireless links and offload tasks to MESs deployed on the BSs. Meanwhile, the BSs are interconnected through high-speed, wired fiber optic connections, which facilitate rapid data transmission with minimal latency. The total number of EDs is represented by $|N|$, and their locations at any time t are denoted by coordinates $(\iota_i^{ED,x}(t), \iota_i^{ED,y}(t))$. The number of BSs is denoted by $|M|$, and their static locations are given by the coordinates $(\iota_j^{BS,x}, \iota_j^{BS,y})$. For analytical convenience and to facilitate subsequent studies, we discretize the entire timeline K into evenly spaced time frames, each of which has a duration denoted by τ. The main parameters are shown in Table I.

TABLE I
Main Parameters and Meaning

Parameters name	Meaning
N	The number of ED
M	The number of BS
$(\iota^x(t), \iota^y(t))$	Device's coordinate position
K	Total time length
τ	slot duration
R	All GPU resource types in IoT system
$f_{j,c}^{es}$	The CPU processing ability of MES j
$f_{j,r,c}^{es}$	The CPU processing ability of the r-th processor
$f_{j,r,g}^{es}$	The GPU processing ability of the r-th processor
$f_{i,c}^{local}$	The CPU processing ability of ED i
$f_{i,r,g}^{local}$	The r-th GPU processing ability of ED i
T_i^t	The task generated by ED i at time t
$\lvert m_i^t \rvert$	The task number generated by ED i at time t
$size_{i,k}^t$	The size of the $T_{i,k}^t$
$C_{i,k}^t$	The CPU resources required to complete $T_{i,k}^t$
$G_{i,k}^t$	The GPU resources required to complete $T_{i,k}^t$
$R_{i,k}^t$	The GPU resource type required for $T_{i,k}^t$
$Q\_device_i(t)$	The size of the awaiting tasks for ED i
$Q\_es_i(t)$	The size of the awaiting tasks for MES j
$Q_{j,r}$	Multi-level queue in the r-th processor at MES j
$A_{ED,i}^t$	The task size processed by ED i at time t
$A_{BS,j}^t$	The task size processed by MES j at time t
$h_{i,j}^t$	The channel gain between ED i and BS j
$t_{i,j}^{tran,t}$	Transmission delay of ED j
$\mathcal{W}_{i,k,j}^{BS,t}$	The waiting times for task $T_{i,k}^t$ at MES j
$\aleph_{i,k}^t$	Systematic completion time for task $T_{i,k}^t$
$Time_{i,k}^t$	Processing delay of task $T_{i,k}^t$

In real IoT systems, the cloud server consists of multiple classes of heterogeneous GPU processing nodes to handle heterogeneous tasks with different resource requirements [11]. Thus, we assume that each BS has a server containing $|R|$ different GPU processing nodes, which can be called the heterogeneous edge server. We denote the type of GPU resources available in the heterogeneous edge server by R, $R = \{0, 1, \ldots, |R|\}$. If $R = r$, it means that besides CPU resources, the processing of heterogeneous computing tasks requires the support of the r-th GPU resources. If $R = 0$, it indicates that the GPU resources required for task processing are generalized, or no specific GPU resources are needed. To simplify

the model complexity, we assume that at MES j, all processors equally share the CPU computing resources, denoted by $f^{es}_{j,c} = \sum_{r=1}^{|R|} f^{es}_{j,r,c}(t)$. $f^{es}_{j,c}$ denotes the total CPU computing resources at MES j, and $f^{es}_{j,r,c}(t)$ denotes the portion of these resources allocated to the r-th processor at time t. Similarly, we denote the GPU processing capability of the r-th processor at time t as $f^{es}_{j,r,g}(t)$.

To address task starvation issues caused by operating systems' inability to accurately predict task processing times, we developed a cloud-based task management framework. This framework establishes multi-level virtual queues for each heterogeneous resource processor, allocating execution time slices based on queue priority. The system continuously monitors task performance and dynamically adjusts priorities, effectively balancing short-term and long-term tasks, enhancing fairness in processing. Detailed information in Section III-A.2.

In the system model, we assume that each ED may generate multiple heterogeneous computing tasks during each time period, with a maximum of Max_n tasks generated in each interval. We use T^t_i to denote the task generated by ED i at time t, $T^t_i = \{T^t_{i,1}, \ldots, T^t_{i,|m^t_i|}\}$. $|m^t_i|$ is the task number generated by ED i, $|m^t_i| \leq Max_n$. $T^t_{i,k} = \{size^t_{i,k}, C^t_{i,k}, G^t_{i,k}, R^t_{i,k}\}$, where $size^t_{i,k}$ indicates the size of the $T^t_{i,k}$, $C^t_{i,k}$ denotes the CPU resources required to complete $T^t_{i,k}$, $G^t_{i,k}$ denotes the GPU resources required to complete $T^t_{i,k}$, and $R^t_{i,k}$ denotes the GPU resource type required for $T^t_{i,k}$. For convenience, we define $size^t_{i,k} = \rho C^t_{i,k}$, where ρ denotes the number of cycles required to process 1 bit of raw data. Additionally, we impose the constraint that $size^t_{i,k} < \omega$, where ω represents the maximum size of each task, we assume ω is a known parameter obtained from previous observations [27].

At time t, ED i generates a task $T^t_{i,k}$. Initially, ED i evaluates whether the task should be offloaded, denoted by $x^t_{i,k} \in \{0, 1\}$. If $x^t_{i,k} = 1$, task $T^t_{i,k}$ will be offloaded to a neighboring MES. Subsequently, the cloud center generates a secondary allocation decision for task $T^t_{i,k}$ of $\{y^t_{i,k}, z^t_{i,k}\}$, determining the specific MES for offloading. Here, $y^t_{i,k} = \{y^t_{i,k,1}, \ldots, y^t_{i,k,|M|}\}$ and $z^t_{i,k} \in \{1, \ldots, |R|\}$. If $y^t_{i,k,j} = 1$, task $T^t_{i,k}$ will be sent to MES j to await processing. Finally, MES j schedules $T^t_{i,k}$ to be processed on processor g based on $R^t_{i,k}$. If $R^t_{i,k} > 0$, $g = R^t_{i,k}$, otherwise $g = z^t_{i,k}$.

A. Task Queue Model

We describe the awaiting queue and heterogeneous resource rotation queue frameworks in the model in this section.

1) Awaiting Task Queue: First, let $Q\_device_i(t)$ denote the size of the awaiting tasks for ED i. Since ED i may generate new tasks in each time frame, $Q\_device_i(t)$ can be dynamically modeled as:

$$Q\_device_i(t+1) = \max\{d^t_i + \tilde{Q}^t_i, 0\}, \quad (1)$$

$$d^t_i = \max\{Q\_device_i(t) - A^t_{ED,i}, 0\}, \quad (2)$$

where $Q\_device_i(0) = 0$. Because the awaiting task size in the queue cannot be negative, the lower bound of $Q\_device_i(t)$ is set to 0, and the following formula is the same. $\tilde{Q}^t_i$ denotes the size of the newly generated tasks that need to be locally processed by ED i at time t, $\tilde{Q}^t_i = \sum_{k=1}^{|m^t_i|}(1 - x^t_{i,k}) size^t_{i,k}$. $A^t_{ED,i}$ denotes the task size processed by ED i at time t. Because CPU involvement is required for the processing of each task, $A^t_{ED,i} \leq \frac{\tau f^{local}_{i,c}}{\rho}$. $f^{local}_{i,c}$ denotes the CPU processing ability of ED i. Meanwhile, $f^{local}_{i,r,g}$ denotes the r-th GPU processing ability of ED i.

Second, we denote the size of the awaiting tasks of BS j by $Q\_es_j(t)$. It can be modeled dynamically as

$$Q\_es_j(t+1) = \max\{b^t_j + \tilde{e}^t_j, 0\}, \quad (3)$$

$$b^t_j = \max\{Q\_es_j(t) - A^t_{BS,j}, 0\}, \quad (4)$$

where $Q\_es_j(0) = 0$. $\tilde{e}^t_j$ denotes the task size offloaded to BS j by each ED at time t, $\tilde{e}^t_j = \sum_{i=1}^{|N|} \sum_{k=1}^{|m^t_i|} x^t_{i,k} y^t_{i,k,j} size^t_{i,k}$. $A^t_{BS,j}$ denotes the task size that can be processed by BS j at time t, $A^t_{BS,j} \leq \frac{\tau f^{es}_{j,c}}{\rho}$.

2) Multi-Level Heterogeneous Resource Feedback Queue: We use Q_j to denote the MHFQ framework at BS j, $Q_j = \{Q_{j,1} \ldots, Q_{j,|R|}\}$. Here, $Q_{j,r}$ represents the set of multi-level feedback queues associated with the r-th processor in the MES j.

$$Q_{j,r} = \{Q^1_{j,r}, Q^2_{j,r}, Q^3_{j,r}\}, \quad (5)$$

where $\{Q^1_{j,r}, Q^2_{j,r}, Q^3_{j,r}\}$ represent the low, medium, and high virtual queues on the r-th processor, respectively. The set $\{\tau^{ves}_1, \tau^{ves}_2, \tau^{ves}_3\}$ represents the time slices allocated by the processing nodes to the tasks in different queues, where $\tau^{ves}_1 < \tau^{ves}_2 < \tau^{ves}_3$. The structure is designed to enable dynamic management of tasks of different sizes. Specifically, smaller tasks are typically completed within the initial time slice τ^{ves}_1. If larger tasks cannot be completed within this initial time slice, they are dynamically transferred to higher-level queues, where they will continue processing from where they were paused. Regardless of which queue a task is in, once it enters the execution phase, it will exclusively use the node's computational resources until the end of its time slice. The specific processing flow is as follows:

When task $T^t_{i,k}$ is offloaded to BS j, it is promptly placed into queue $Q^1_{j,r}$ based on $R^t_{i,k}$ and $z^t_{i,k}$, with the entry time noted as $to^1_{j,r}(T^t_{i,k})$. When the task reaches the top of this queue, marked at $top^1_{j,r}(T^t_{i,k})$, it begins processing, with a maximum allowable period of τ^{ves}_1. If the task is not completed within this period, it is paused and moved to queue $Q^2_{j,r}$ for further waiting, where the transfer time is recorded as $to^2_{j,r}(T^t_{i,k})$. Processing resumes from the paused point when the task reaches the top of $Q^2_{j,r}$. If further processing is needed, the task is transferred to $Q^3_{j,r}$ and is treated as a complex task, with no further rotation required. Where, $to^q_{j,r}(T^t_{i,k})$ denotes the time when the task reaches the queue $Q^q_{j,r}$ and $top^q_{j,r}(T^t_{i,k})$ denotes the time when the task reaches the top of queue $Q^q_{j,r}$. This level-by-level migration task management mechanism effectively reduces the idle time of the processing node's computing resources and improves the system's task processing performance and throughput.

B. Communications Model

Considering that the quality of task transmission is easily affected by obstacles or reflections, we use $h^t_{i,j}$ to represent

the channel gain between ED i and BS j.

$$h_{i,j}^t = A_{antenna}\left(3 * \frac{10^8}{4\pi f_{carrier} dis_{i,j}^t}\right)^{loss_{ple}}, \tag{6}$$

$$dis_{i,j}^t = \sqrt{\left(\iota_i^{ED,x}(t) - \iota_j^{BS,x}\right)^2 + \left(\iota_i^{ED,y}(t) - \iota_j^{BS,y}\right)^2}, \tag{7}$$

where $A_{antenna}$ denotes the antenna gain, $f_{carrier}$ denotes the carrier frequency, $dis_{i,j}^t$ denotes the distance between ED i and BS j, $loss_{ple}$ denotes the path attenuation index. We set $h_{i,j}^t$ constant within a single time slot, but varying between time slots. Then, we can derive the transmission rate $v_{i,j}^t$:

$$v_{i,j}^t = B_i^t * \log_2\left(1 + \frac{P_i^{tran} h_{i,j}^t}{\sigma^2}\right), \tag{8}$$

where B_i^t denotes the channel bandwidth, P_i^{tran} denotes the transmit power of ED i, and σ^2 denotes Gaussian white noise. Transmission delay $t_{i,j}^{tran,t}$:

$$t_{i,j}^{tran,t} = \frac{o_u(t) size_{i,k}^t}{v_{i,j}^t}, \tag{9}$$

where $o_u(t) > 1$, is the additional information overhead during data transfer [27]. In conclusion, we can obtain the offload completion time $to_{j,r}^1(T_{i,k}^t)$ recorded by the IoT system, when task $T_{i,k}^t$ is offloaded to the MES j and enters the queue $Q_{j,r}^1$ of r-th processor.

$$to_{j,r}^1(T_{i,k}^t) = t_{i,j}^{tran,t} + t. \tag{10}$$

where t represents the specific time when the task $T_{i,k}^t$ is generated within the ED i.

C. Computational Model

1) Local Computation: When task $T_{i,k}^t$ is processed locally in the ED i, task $T_{i,k}^t$ exclusively occupies the computing resources. The processing delay is:

$$\mathcal{D}_{i,k}^{ED,t} = \max\left\{\frac{C_{i,k}^t}{f_{i,c}^{local}}, \frac{G_{i,k}^t}{f_{j,r,g}^{local}}\right\}, \tag{11}$$

$$f_{i,r,g}^{local} = \begin{cases} f_{i,R_{i,k}^t,g}^{local}, & \text{if } R_{i,k}^t > 0 \\ f_{i,z_{i,k}^t,g}^{local}, & \text{if } R_{i,k}^t = 0 \end{cases} \tag{12}$$

Task $T_{i,k}^t$ is deemed complete only when both its GPU and CPU components have been fully processed. Consequently, the completion time at which task $T_{i,k}^t$ completes, denoted as $\aleph_{i,k}^{ED,t}$, can be determined as follows:

$$\aleph_{i,k}^{ED,t} = \mathcal{D}_{i,k}^{ED,t} + top_i^{local}(T_{i,k}^t), \tag{13}$$

where, $top_i^{local}(T_{i,k}^t)$ represents the time when task $T_{i,k}^t$ arrives at the top of the ED i queue. Furthermore, $top_i^{local}(T_{i,k+1}^t) = \aleph_{i,k}^{ED,t}$.

2) MES Computation: When task $T_{i,k}^t$ is offloaded for MES processing, we first determine the CPU and GPU processing delays ($\mathcal{D}_{i,k,j}^{c,t}$, $\mathcal{D}_{i,k,j}^{g,t}$) of $T_{i,k}^t$ in the MHFQ:

$$\mathcal{D}_{i,k,j}^{c,t} = \sum_{q=1}^{2} \min\left\{\frac{\max\{C_{i,k}^t - \sum_{p=0}^{q-1}\gamma_{j,c}^t(p), 0\}}{f_{j,r,c}^{es}(\lfloor top_{j,r}^q(T_{i,k}^t)\rfloor)}, \tau_q^{ves}\right\}$$
$$+ \frac{\max\{C_{i,k}^t - \sum_{p=0}^{2}\gamma_{j,c}^t(p), 0\}}{f_{j,r,c}^{es}(\lfloor top_{j,r}^3(T_{i,k}^t)\rfloor)}, \tag{14}$$

$$\mathcal{D}_{i,k,j}^{g,t} = \sum_{q=1}^{2} \min\left\{\frac{\max\{G_{i,k}^t - \sum_{p=0}^{q-1}\gamma_{j,g}^t(p), 0\}}{f_{j,r,g}^{es}(\lfloor top_{j,r}^q(T_{i,k}^t)\rfloor)}, \tau_q^{ves}\right\}$$
$$+ \frac{\max\{G_{i,k}^t - \sum_{p=0}^{2}\gamma_{j,g}^t(p), 0\}}{f_{j,r,g}^{es}(\lfloor top_{j,r}^3(T_{i,k}^t)\rfloor)}, \tag{15}$$

where $f_{j,r,g}^{es}(\lfloor top_{j,r}^q(T_{i,k}^t)\rfloor)$ and $f_{j,r,c}^{es}(\lfloor top_{j,r}^3(T_{i,k}^t)\rfloor)$ denote the computational power available when the task $T_{i,k}^t$ reaches the top of the queue $Q_{j,r}^q$. Similar $f_{i,r,g}^{local}$, when $R_{i,k}^t > 0$, $r = R_{i,k}$. Otherwise, $r = z_{i,k}^t$. $\gamma_{j,c}^t(p)$ and $\gamma_{j,g}^t(p)$ respectively represent the processed CPU and GPU task sizes in p-level queues, with $\gamma_{j,c}^t(0) = \gamma_{j,g}^t(0) = 0$. Summarizing, we can obtain:

$$\mathcal{D}_{i,k,j}^{BS,t} = \max\left(\mathcal{D}_{i,k,j}^{c,t}, \mathcal{D}_{i,k,j}^{g,t}\right). \tag{16}$$

In addition to the processing delays, the waiting times in each level queue for task $T_{i,k}^t$ are also significant and should not be overlooked.

$$\mathcal{W}_{i,k,j}^{BS,t} = \sum_{q=1}^{3}\left(\prod_{p=0}^{q-1} SC_{i,k,j}^{t,p}\right)(top_{j,r}^q(T_{i,k}^t) - to_{j,r}^q(T_{i,k}^t)), \tag{17}$$

where, $SC_{i,k,j}^{t,p}$ indicates whether task $T_{i,k}^t$ is successfully completed at p-level queue. If the task is completed, then $SC_{i,k,j}^{t,p} = 0$, otherwise, $SC_{i,k,j}^{t,p} = 1$. If task $T_{i,k}^t$ is offloaded to the MES for processing, the task completion delay can be expressed as:

$$\aleph_{i,k}^{BS,t} = \sum_{j=1}^{M} y_{i,k,j}^t(\mathcal{W}_{i,k,j}^{BS,t} + \mathcal{D}_{i,k,j}^{BS,t} + to_{j,r}^1(T_{i,k}^t)). \tag{18}$$

Based on the above, the system delay for task $T_{i,k}^t$ completion is $\aleph_{i,k}^t = (1 - x_{i,k}^t)\aleph_{i,k}^{ED,t} + x_{i,k}^t\aleph_{i,k}^{BS,t}$, the task processing delay is $Time_{i,k}^t = \aleph_{i,k}^t - t$.

D. Problem Programming

We investigate the problem of online processing for large-scale, randomly generated heterogeneous computing tasks. Our goal is to enhance the processing efficiency of IoT systems, measured by the average task processing size per unit time ($[P1]$). To achieve this goal, the system dynamically makes offloading decisions for newly generated tasks in each time period. We formulate this issue as a multi-stage mixed-integer nonlinear programming (MINLP) problem:

$$[P1]: \max_{x^t, y^t, z^t} \lim_{K\to\infty} \frac{1}{K}\sum_{t=1}^{K}\sum_{i=1}^{|N|}\sum_{k=1}^{|m_i^t|}\frac{size_{i,k}^t}{Time_{i,k}^t}, \tag{19}$$

$$\text{s.t. } \lim_{K\to\infty}\frac{1}{K}\sum_{t=1}^{K}\mathbb{E}[Q\_device_i(t)] < \infty, \tag{19a}$$

$$\lim_{K\to\infty}\frac{1}{K}\sum_{t=1}^{K}\mathbb{E}[Q\_es_i(t)] < \infty, \tag{19b}$$

$$top_{j,r}^q(T_{i,k}^t) - to_{j,r}^q(T_{i,k}^t) > 0, \tag{19c}$$

$$A^t_{ED,i} \le \frac{\tau f^{local}_{i,c}}{\rho}, A^t_{BS,j} \le \frac{\tau f^{es}_{j,c}}{\rho}, \tag{19d}$$

$$x^t_{i,k} \in \{0,1\}, y^t_{i,k} \in \{1,2,\ldots,|N|\}, \tag{19e}$$

$$z^t_{i,k} \in \{1,2,\ldots,|R|\}, \tag{19f}$$

$$\omega < \infty, |m^t_i| \le Max_n, \tag{19g}$$

where constraints (19a) and (19b) denote ED and MES queue fluctuations. Constraints (19c) and (19d) represent time and processing capacity constraints on the task's processing. Constraints (19e) and (19f) denote the range of task offloading decisions. Constraints (19g) denotes the state constraints for newly generated tasks at the ED during each frame. Problem [P1] is a multi-stage MINLP problem. The complexity and high dimensionality of this problem make it exceedingly challenging to solve directly. In the next section, we employ the Lyapunov optimization method to decouple this problem into deterministic MINLP subproblems for each time frame.

IV. Decoupling of MINLP Problems Based on Lyapunov Optimization

To remove the constraints (19a), (19b), we define $Z(t) = \{Q\_device(t), Q\_es(t)\}$. $Q\_device(t) = \{Q\_device_i(t)\}^{|N|}_{i=1}$, $Q\_es(t) = \{Q\_es_j(t)\}^{|M|}_{j=1}$. The Lyapunov function effectively assesses system states over an infinite time horizon and ensures global stability. Therefore, we utilize this method to maintain the stability of the task queue $\{Q\_device(t), Q\_es(t)\}$, and achieve the joint optimization of the task processing speed and the system load.

Based on this, we can construct the Lyapunov function $L(Z(t))$ and the Lyapunov drift function $\Delta L(Z_t)$:

$$L(Z(t)) = \frac{1}{2}(\sum_{i=1}^{N} Q^2\_device_i(t) + \sum_{j=1}^{M} Q^2\_es_j(t)), \tag{20}$$

$$\Delta L(Z_t) = \mathbb{E}\{L(Z(t+1)) - L(Z(t))|\mathbf{Z(t)}\}. \tag{21}$$

To ensure the stability of the $Z(t)$ while maximizing the long-term task processing capability of the IoT system, we employ the drift-plus-penalty method to minimize problem [P1].

$$\min \wedge Z(t) = \Delta L(Z_t) - V \sum_{i=1}^{|N|} \mathbb{E}\left\{ \sum_{k=1}^{|m^t_i|} \frac{size^t_{i,k}}{Time^t_{i,k}} |\mathbf{Z(t)} \right\}, \tag{22}$$

where, $V > 0$ is a penalty factor that quantifies the importance of the penalty term. From this formulation, minimizing the drift term effectively enhances the stability of the IoT system's queue. However, this approach may not yield the optimal solution to problem [P1]. Conversely, the value of V that minimizes problem [P1] might not guarantee the stability of the IoT system's queue. Therefore, the $\wedge Z(t)$ effectively balances both queue stability and the objective function. We approximate the minimum of equation (22) by iteratively minimizing the upper bound of $\wedge Z(t)$.

$$\begin{aligned}(Q\_device_i(t+1))^2 &\le (d^t_i)^2 + (\widetilde{Q}^t_i)^2 + 2d^t_i\widetilde{Q}^t_i \\ &\le Q\_device_i(t)^2 + (A^t_{ED,i})^2 + (\widetilde{Q}^t_i)^2 \\ &\quad + 2Q\_device_i(t)(\widetilde{Q}^t_i - A^t_{ED,i}),\end{aligned} \tag{23}$$

$$\begin{aligned}(Q\_es_j(t+1))^2 &\le (Q\_es_j(t))^2 + (\widetilde{e}^t_j)^2 + (A^t_{BS,j})^2 \\ &\quad + 2Q\_es_j(t)(\widetilde{e}^t_j - A^t_{BS,j}).\end{aligned} \tag{24}$$

We set

$$L(Q\_device(t)) = \frac{1}{2}\sum_{i=1}^{N} Q^2\_device_i(t)), \tag{25}$$

$$\begin{aligned}\Delta L(Q\_device(t)) = \mathbb{E}\{&L(Q\_device(t+1)) \\ &- L(Q\_device(t))|\mathbf{Z(t)}\}.\end{aligned} \tag{26}$$

Similarly, we can get

$$L(Q\_es(t)) = \frac{1}{2}\sum_{j=1}^{M} Q^2\_es_j(t)), \tag{27}$$

$$\Delta L(Q\_es(t)) = \mathbb{E}\{L(Q\_es(t+1)) - L(Q\_es(t))|\mathbf{Z(t)}\}. \tag{28}$$

Summing the queues on both sides of the equation (23) (24) cumulatively, and taking its expected value, we can get

$$\Delta L(Q\_device(t)) \le \sum_{i=1}^{N} \mathbb{E}\{Q\_device_i(t)(\widetilde{Q}^t_i - A^t_{ED,i})\} + UP_1, \tag{29}$$

$$UP_1 = \frac{1}{2}\sum_{i=1}^{N} (\omega Max_n)^2 + \left(\frac{\tau f^{local}_{i,c}}{\rho}\right)^2. \tag{30}$$

And,

$$\Delta L(Q\_es(t)) \le \sum_{j=1}^{M} \mathbb{E}\{Q\_es_j(t)(\widetilde{e}^t_j - A^t_{BS,j})\} + UP_2, \tag{31}$$

$$UP_2 = \frac{1}{2}\left(\sum_{j=1}^{M} (\frac{\tau f^{es}_{j,c}}{\rho})^2 + (|N|\omega Max_n)^2\right). \tag{32}$$

Therefore, we can get the upper term of equation (22),

$$\begin{aligned}&UP + \sum_{i=1}^{|N|} Q\_device_i(t)\mathbb{E}\{\widetilde{Q}^t_i - A^t_{ED,i}|\mathbf{Z(t)}\} \\ &\quad + \sum_{j=1}^{|M|} Q\_es_j(t)\mathbb{E}\{\widetilde{e}^t_j - A^t_{BS,j}|\mathbf{Z(t)}\} \\ &\quad - V\sum_{i=1}^{|N|}\mathbb{E}\left\{\sum_{k=1}^{|m^t_i|}\frac{size^t_{i,k}}{Time^t_{i,k}}|\mathbf{Z(t)}\right\}\end{aligned}$$

where $UP = UP_1 + UP_2$. After eliminating irrelevant terms and rearranging, we obtain problem [P2]:

$$\begin{aligned}[P2]: \max_{x^t,y^t,z^t} &V\sum_{i=1}^{|N|}\mathbb{E}\left[\sum_{k=1}^{|m^t_i|}\frac{size^t_{i,k}}{Time^t_{i,k}}|\mathbf{Z(t)}\right] \\ &- \sum_{i=1}^{|N|} Q\_device_i(t)\mathbb{E}[\widetilde{Q}^t_i|\mathbf{Z(t)}] \\ &- \sum_{j=1}^{|M|} Q\_es_j(t)\mathbb{E}[\widetilde{e}^t_j|\mathbf{Z(t)}],\end{aligned} \tag{33}$$

s.t. (19a), (19b), (19e), (19f). (33a)

In Section IV, we decouple problem [P1], but problem [P2] is a nonconvex optimization problem, and the coupling between variables makes it extremely challenging to solve [P2] directly. In Section V, we design a new algorithm to solve the problem [P2] under each frame in an online manner.

V. Multi-Agent DRL-Based Task Real-Time Decision Algorithm

Although Lyapunov optimization provides a theoretical foundation for addressing problems across time intervals, it requires a comprehensive and accurate mathematical description of system dynamics. Achieving this is challenging in IoT environments that feature numerous interacting entities. In contrast, multi-agent deep reinforcement learning (DRL) makes quick decisions based on the current state, which is performed based on the Markov decision process (MDP). However, IoT environments are non-Markovian, which means that traditional DRL methods cannot adapt to the complexity of such environments, limiting their effectiveness. By integrating the stability benefits of Lyapunov methods in non-Markovian environments with the decision-making flexibility of reinforcement learning, we can effectively address the dynamic complexity of IoT environments with multiple interacting agents and non-linear changes.

A. Problem Analysis

In practical IoT systems, each ED and the Cloud can be considered as independent agents. Specifically, each ED determines whether to offload tasks based on its observable partial state and predicted future task arrival states. After all tasks have been offloaded from the EDs, the Cloud agent takes over the management, dynamically allocating tasks to the suitable MESs based on the real-time states of the MESs, the states of tasks offloaded in the current time slot, and the predicted offloading states of tasks for the next time slot.

This decentralized decision-making process necessitates robust coordination mechanisms to ensure efficient task distribution and system stability. Then, we have developed a multi-agent DRL framework. Within this framework, each agent, whether an ED or a Cloud agent, is designed to make decisions that not only maximize its own task processing efficiency but also contribute to the overall performance of the system. Consequently, based on these differing roles and attributes, the problem [P2] is decomposed into subproblems [P3] and [P4].

$$[P3]: \max_{\{x^t,z^t\}} V\sum_{k=1}^{|m_i^t|}(1-x_{i,k}^t)\frac{size_{i,k}^t}{Time_{i,k}^t}-Q\_device_i(t)\widetilde{Q}_i^t,$$

$$[P4]: \max_{\{y^t,z^t\}} \sum_{j=1}^{|M|}\left(V\sum_{i=1}^{|N|}\sum_{k=1}^{|m_i^t|}x_{i,k}^t y_{i,k,j}^t\frac{size_{i,k}^t}{Time_{i,k}^t}-Q\_es_j(t)\widetilde{e}_j^t\right).$$

Specifically, the problem [P3] defines the objective function for each ED agent, wherein offloading decisions $\{x_i^t, z_i^t\}$ for each task are made by maximizing [P3]. Problem [P4] establishes the objective function for the cloud agent, which identifies the optimal allocation strategies $\{y^t, z^t\}$ for tasks offloaded to the MESs by maximizing [P4]. By collectively making appropriate processing decisions for newly generated tasks in each time frame, these agents effectively minimize fluctuations in the queues of each device (ED and MES) and enhance the task processing capabilities of the IoT system.

B. LRMA Algorithm

Deep reinforcement learning discretizes time and enables the agent to interact with the MES sequentially across time frames, aiming to learn the optimal task offloading strategy. Let $\mathcal{S}(t)$ denote the local state observable by the agent, $\mathcal{S}(t) = \{\mathcal{S}_1(t), \ldots, \mathcal{S}_{|N|}(t), \mathcal{S}^{cloud}(t)\}$. As illustrated in Fig. 2, at time t, each agent selects an appropriate action $a_i(t)$ from the action space A based on the observable state. The IoT environment evaluates the joint action $a(t)$ according to the joint action and reward value function, thereby selecting the optimal joint action $a^*(t)$, $a^*(t) = \{a_1^*(t), \ldots, a_N^*(t)\}$. Finally, based on the collected optimal state-action pairs, each agent's Actor network is updated until convergence to the optimal policy.

1) State Space: The state serves as a representation reflecting the conditions of the IoT environment, helping the agent to better understand and adapt to the complex IoT landscape. At time t, we denote the observable local state of ED i when making decisions for task $T_{i,k}^t$ as $\mathcal{S}_{i,k}(t)$:

$$\mathcal{S}_{i,k}(t) = \left\{T_{i,k}^t, \widetilde{T}_{i,k^-}^t, Q\_device_i(t), \sum_{j=1}^{n} Q\_es_j(t), \beta_i^{t+1}\right\}, \tag{34}$$

$\widetilde{T}_{i,k^-}^t$ denotes the state of other awaiting decision-making tasks, $\widetilde{T}_{i,k^-}^t = \{|m_i^t| - 1, \sum_{k^- \neq k}\{C_{i,k^-}^t, G_{i,k^-}^t\}, \mu_{i,k}^t\}$. $\mu_{i,k}^t$ is a vector of $1 * |R|$, where $\mu_{i,k}^t[r]$ denotes the task number of GPU type r within ED i, excluding $T_{i,k}^t$. $\sum_{j=1}^{n} Q\_es_j(t)$ denotes the pending task size of the n MESs near ED i. β_i^{t+1} represents the next-time prediction state obtained by LSTM.

Similar to the ED agent, we use $\mathcal{S}_{i,k}^{cloud}(t)$ to denote the observable state when the cloud agent makes secondary allocation decisions for the offloaded task $T_{i,k}^t$:

$$\mathcal{S}_{i,k}^{cloud}(t) = \{T_{i,k}^t, \varsigma_{k^-}^t, \{Q\_es_j(t)\}_{j=1}^n, \beta_{cloud}^{t+1}\}, \tag{35}$$

Similar to $\widetilde{T}_{i,k^-}^t$, $\varsigma_{k^-}^t$ represents the states of other pending decision-making tasks in the MES side, excluding the task $T_{i,k}^t$. $\{Q\_es_j(t)\}_{j=1}^n$ are the specific state of N MESs. β_{cloud}^{t+1} denotes the prediction state of the Cloud for the next-time.

2) Action Space: In DRL, action is a decision taken by the agent's based on the observable state. In order to maximize the immediate return, the ED agent i takes action based on the system state $\mathcal{S}_{i,k}(t)$, deciding whether the task $T_{i,k}^t$ should be executed locally or offloaded. Thus the action space $a_{i,k}(t)$ of ED i in processing task $T_{i,k}^t$ is denoted as

$$a_{i,k}(t) = \{x_{i,k}^t, z_{i,k}^t\}. \tag{36}$$

If $x_{i,k}^t = 0$, task $T_{i,k}^t$ is processed locally. If $x_{i,k}^t = 1$, task $T_{i,k}^t$ is offloaded to the edge server. Based on the decisions made by the ED agent where $x_{i,k}^t = 1$ denotes the offloaded task $T_{i,k}^t$, the cloud makes secondary allocation decisions to determine which MES the task $T_{i,k}^t$ should be assigned to. The action space $a_{i,k}^{cloud}(t)$ is:

$$a_{i,k}^{cloud}(t) = \{y_{i,k}^t, z_{i,k}^t\}. \tag{37}$$

$y_{i,k}^t$ indicates to which MES task $T_{i,k}^t$ should be allocated. If task $T_{i,k}^t$ is a CPU or general-purpose GPU task ($R_{i,k}^t = 0$), $z_{i,k}^t$

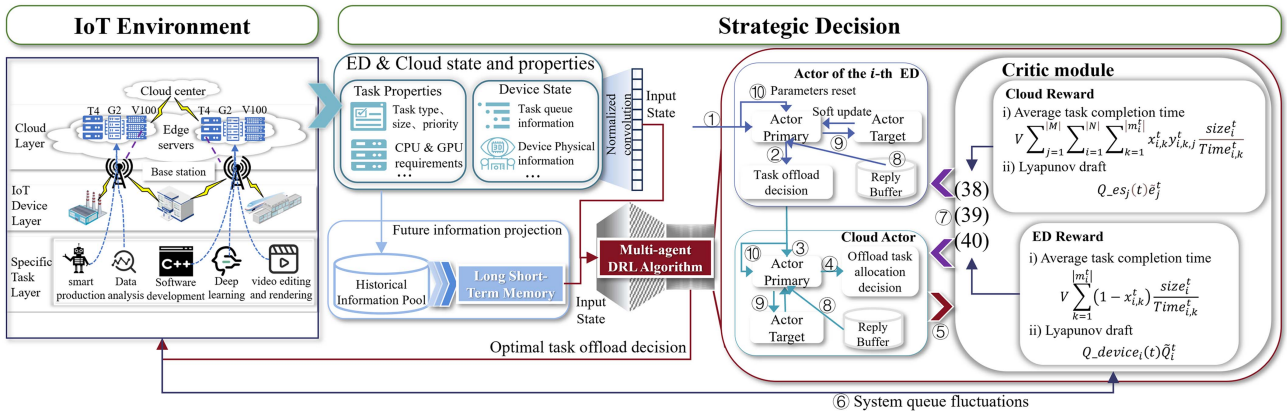

Fig. 2. Flowchart of multi-agent reinforcement learning algorithm based on LSTM and parameter reset.

denotes which GPU processing node at MES $y_{i,k}^t$ the task $T_{i,k}^t$ should be queued for processing.

3) Reward: Based on the joint action $a(t)$, agents receive immediate rewards from the environment to learn the LRMA strategy. However, DRL algorithms aim to maximize immediate rewards, which may lead agents to frequently try new strategies that rapidly increase system load or resource usage—potentially exceeding system capacity and degrading overall performance.

To address this issue, we incorporated a Lyapunov term into the immediate reward function, integrating system stability into the agents' decision-making to balance short-term and long-term objectives. By considering Lyapunov drift, agents focus not only on immediate gains but also on the long-term impact of their actions on system stability. The immediate reward in LRMA comprises two components: team reward and individual reward. The individual reward for ED i is derived from problem $[P3]$,

$$r_i^t = V\sum_{k=1}^{|m_i^t|}(1-x_{i,k}^t)\frac{size_i^t}{Time_{i,k}^t} - Q\_device_i(t)\widetilde{Q}_i^t. \quad (38)$$

The individual reward $r_0(t)$ for Cloud agent is derived from problem $[P4]$.

$$r_0^t = \sum_{j=1}^{|M|}\left(V\sum_{i=1}^{|N|}\sum_{k=1}^{|m_i^t|}x_{i,k}^t y_{i,k,j}^t\frac{size_i^t}{Time_{i,k}^t} - Q\_es_j(t)\widetilde{e}_j^t\right). \quad (39)$$

The team rewards $r_{all}^t = \sum_{i=0}^{|N|} r_i^t$. To comprehensively evaluate the relationship between individual reward and team cooperation, we use $r_i^{tot}(t)$ to denote the comprehensive reward obtained by agent i for taking action $a_i(t)$.

$$r_i^{tot} = \alpha r_i^t + (1-\alpha)r_{all}^t. \quad (40)$$

where, α represents the weighting factor, used to balance team collaboration and individual reward, $\alpha \in (0,1)$.

C. LRMA Algorithm Design

The proposed LRMA algorithm framework[1], as shown on Fig. 2, consists of two modules for each agent: the Actor module and the Critic module. The Actor module includes two networks: the Actor's primary and target networks. The Actor primary network makes decisions based on the local observation states, while the Critic module comprehensively evaluates the Actor module's actions. To mitigate the risk of overfitting in DRL algorithms, a periodic resetting of Actor parameters alleviates the primacy bias [21]. In the LRMA algorithm, we utilize a centralized training and distributed execution approach. During the centralized training phase, the Actor module generates task-processing decisions. In contrast, the Critic module evaluates based on all agents' joint actions. The Critic module doesn't participate in the distributed execution phase, where each agent only requires an Actor-network.

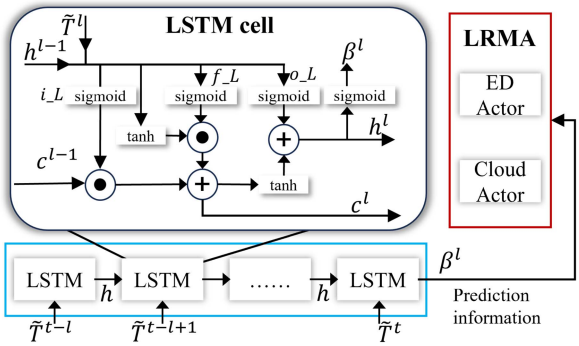

Fig. 3. LSTM network flowchart.

1) LSTM Network: The LSTM is a specialized type of recurrent neural network that incorporates memory units and gating mechanisms (input i_L, forget f_L, and output o_L gates). It learns from the historical task arrival states $\widetilde{T} = \{\widetilde{T}^{t-l}, \widetilde{T}^{t-l+1}, \ldots, \widetilde{T}^t\}$ and predicts future states β^t, such as the number, type and resource requirements of tasks that may arrive in the next time slot. This supports ED and Cloud Actors in making informed decisions within current time. As shown in Fig. 3. h^{l-1} represents the hidden state of the previous time step. c^{l-1} represents the memory unit state from the previous time step, which is used to store and transfer long-term dependent

[1]The initial version of the algorithm, the dataset, and the corresponding description are available on GitHub:https://github.com/HE-xiao4333/TNSE-Large-scale-Heterogeneous-Task-Offloading.

information.

$$i\_L^t = sigmod\left(W_{il}\left[h^{t-1}, \widetilde{T}^t\right] + b_{il}\right),$$

$$f\_L^t = sigmod\left(W_{fl}\left[h^{t-1}, \widetilde{T}^t\right] + b_{fl}\right),$$

$$o\_L^t = sigmod\left(W_{ol}\left[h^{t-1}, \widetilde{T}^t\right] + b_{ol}\right),$$

$$c^t = f\_L^t \cdot c^{t-1} + i\_L^t \cdot tanh\left(W_l\left[h^{t-1}, \widetilde{T}^l\right] + b_l\right),$$

$$h^t = o\_L^t \cdot tanh(c^t), \beta^t = softmax(W_y h^t + b_y), \quad (41)$$

where, W and b represent the weights and parameters of each layer, respectively. The network is trained using the Adam optimizer. The loss function is defined as the difference between actual and predicted information.

$$Loss_{lstm} = \frac{1}{N+1}\sum_{i=1}^{N+1}\left(\widetilde{T}^t - \beta^t\right)^2. \quad (42)$$

2) Actor Module: Following the predictions provided by the LSTM, Actor h makes offloading and allocation decisions for task $T_{h,k}$ based on the predicted information β_h and the observed local state $S_{h,k}$. The policy of ED Actor network is denoted as $\pi^{ed} = \{\pi^{ed}_{\theta_1}, \ldots, \pi^{ed}_{\theta_{|N|}}\}$, where θ_h represents the policy parameters of Actor h. The policy for the Cloud Actor network is represented as $\pi^{cloud}_{\theta_{|N+1|}}$. The combined policy $\pi = \{\pi^{dev}, \pi^{mes}\}$ enables the Actor module to make appropriate actions $a(t) = \{a_{h,k}(t), a^{cloud}_{h,k}(t)\}$ for each task based on the states $\{\mathcal{S}_{h,k}(t), \mathcal{S}^{cloud}_{h,k}(t)\}$.

$$\pi^{dev}_{\theta^t_h} : \{\mathcal{S}_{h,k}(t) \rightarrow a_{h,k}(t)\},$$

$$\pi^{cloud}_{\theta_{|N|+1}} : \{\mathcal{S}^{cloud}_{h,k}(t) \rightarrow a^{cloud}_{h,k}(t)\}. \quad (43)$$

To enhance the action space, we employ a point-to-uniform variation method to quantize $a(t)$, obtaining multiple candidate solutions $a^t_P = \{a_1(t), \ldots, a_P(t)\}$.

3) Critic Module: During the centralized training phase, the LRMA algorithm evaluates the joint action $a(t)$ formed by the combination of actions using formulas (38), (39), and (40). Consequently, the expected reward brought by the joint action for agent h is represented as follows:

$$\mathcal{T}(\theta_h, a(t)) = \mathbb{E}[r^{tot}_h | \mathcal{S}^t, \{a_{i,k}(t), a^{cloud}_{i,k}(t)\}^{|N|,|m^t_i|}_{i,k=1,1}]. \quad (44)$$

Based on the expected reward, the optimal action $a^\star_h(t)$ is selected for device h within the current time, $a^\star_h(t) = argmax\mathcal{T}(a^t_P)$.

4) Strategy Update Module: After obtaining the best state action pair, we store it in the reply buffer. The reply buffer includes the current state, selected action, and reward. To mitigate the impact of data dependency, experience data is sampled in batches to train both the primary and target networks, with a batch size denoted as $\mathcal{B}$. The policy parameters of agent h are updated using the Adam optimization algorithm, with periodic resets implemented. Compared to the Actor primary network, the Actor target network undergoes delayed updates to stabilize and optimize the training process in DRL. Parameters are periodically refreshed from the Actor primary network through a soft update mechanism.

$$\pi^{target}_{\theta_h} = \xi^{soft}\pi^{primary}_{\theta_h} + (1-\xi^{soft})\pi^{target}_{\theta_h}. \quad (45)$$

Where ξ^{soft} denotes the soft update factor. The details of the algorithm are shown in Algorithm 1.

Algorithm 1: LRMA Algorithm Train Framework.

1: **Input:** Parameters N, M, K, V, ω, ξ^{soft}, $f^{local}_{i,r,g}$, $f^{local}_{i,c}$, $\mathcal{B}$, soft update interval δ^{soft}, parameter reset slot δ^{reset};
2: Initialize the reply buffer $\{\mathcal{R}^{ED}, \mathcal{R}^{Cloud}\}$, LSTM parameter θ^{lstm}, and Actor parameter $\{\theta^{primary}, \theta^{target}\}$;
3: Load task dataset $T_{task} = \{T^1_i, \ldots, T^K_i\}^N_{i=1}$;
4: **For** $t = \{1, 2, \ldots, K\}$ **do**
5: **For** $i = \{1, 2, \ldots, N\}$ **do**
6: Update θ^{lstm} based on previous state β^{t-1}_i, T^t_i;
7: Predicting state β^t_i using LSTM;
8: Get state $\mathcal{S}_i(t)$ from β^t_i and IoT system;
9: **For** $T^t_{i,k} \in T^t_i$ **do**
10: Get decision $a_{i,k}(t)$ using $\mathcal{S}_{i,k}(t)$ by Actor i;
11: $a_i(t) \leftarrow a_{i,k}(t)$;
12: $T^t_{off} \leftarrow \{T^t_{i,k} | x^t_{i,k} == 1\}$;
13: **EndFor**
14: update $a_{ed}(t) \leftarrow a_i(t)$;
15: **EndFor**
16: **For** $T^t_{i,k} \in T^t_{off}$ **do**
17: Get cloud state $\mathcal{S}^{cloud}_{i,k}(t)$;
18: Get action $a^{cloud}_{i,k}(t)$ and update $a_{cloud}(t)$;
19: **EndFor**
20: Update joint actions set $a^t_P \leftarrow \{a_{ed}(t), a_{cloud}(t)\}$;
21: Getting the optimal joint action $a^*(t)$ by equation (38), (39), (40);
22: Update $\mathcal{R}^{ED}, \mathcal{R}^{Cloud}$ by $\{\mathcal{S}(t), a^*(t)\}$;
23: **IF** $mod(t, \delta^{soft}) = 0$ **do**
24: Randomly and uniformly sample $\mathcal{B}$ batch from $\{\mathcal{R}^{ED}, \mathcal{R}^{Cloud}\}$;
25: Update $\{\theta^{primary}, \theta^{target}\}$ by soft-update ξ^{soft};
26: **IF** $mod(t, \delta^{reset}) = 0$ **do**
27: Reset Actor primary network's last layer parameters;
28: **EndIF**
29: **EndIF**
30: Perform task processing and update IoT system status based on $a^*(t)$;
31: $t \leftarrow t + 1$;
32: **EndFor**

In the initial phase of the algorithm, system and algorithmic parameters are set (lines 1-2). Subsequently, we load the task dataset $T_{task} = \{T^1_i, \ldots, T^K_i\}^N_{i=1}$, where T^t_i denotes the set of heterogeneous tasks arriving at time for ED i, represented as $T^t_i = \{T^t_{i,1}, \ldots, T^t_{i,|m^t_i|}\}$. Each task $T^k_{i,k}$ is randomly selected from dataset [11] for four different classes of heterogeneous tasks with different requirements and characterized by generation time, originating device, task size, task type, and the CPU, GPU computational requirements (line 3). At the beginning of time t, the ED i predicts future environment arrivals state β^t_i based on historical states using the LSTM model (lines 6-7). The β^t_i combined with the local environmental information observed by the ED i, forms the basic state space $\mathcal{S}_i(t)$ for task

offloading decisions (line 8). Subsequently, ED generates the respective state space $\mathcal{S}_{i,k}(t)$ for each pending decision task, where $\mathcal{S}_{i,k}(t) = \{\mathcal{S}_i(t), T^t_{i,k}, \widetilde{T}^t_{i,k^-}\}$. Based on $\mathcal{S}_{i,k}(t)$, ED i determines whether the task is to be executed locally or offloaded to the edge cloud (lines 9–13). The state information of all the offloaded tasks is aggregated into the offloading matrix T^t_{off}. The cloud center then makes an allocation decision $a^{cloud}_{i,k}(t)$ for each task within T^t_{off} based on the queue state $Q\_es_j(t)$ of each server and the task state $T^t_{i,k}$, among other factors, to ensure that each task is assigned to the appropriate processing node (lines 16–19). Next, the algorithm selects the optimal solution $a^*(t)$ that maximizes the immediate reward within the current time using (38), (39), and (40), and updates the reply buffer both the ED and the cloud $\mathcal{R}^{ED}, \mathcal{R}^{Cloud}$ (lines 20–23). After a certain period, the system trains the corresponding Actor network based on $\mathcal{R}^{ED}, \mathcal{R}^{Cloud}$, and resets the parameters after δ^{reset} rounds of training (lines 24–31). Finally, the IoT system performs task offloading and processing based on the optimal solution $a^*(t)$ (lines 32–34).

D. Time Complexity Analysis

In this section, we analyze the time complexity of the LRMA algorithm. The main execution of the LRMA algorithm in each time slot includes task offloading decision at the ED and task allocation decision at the cloud. During the network training process, optimal decision actions are obtained by training the network at each time frame. First, we derive the time complexity of the LRMA training algorithm. The Actor primary and target networks consist of 1 input layer, 2 hidden layers, and 1 output layer, all of which are fully connected layers. Let g^a_l represent the number of neurons in the l-th layer of the Actor network. Since the target network only periodically updates the parameters of the 'primary-network' without training, its time complexity can be ignored. According to [34], we can calculate the total computational complexity for training N agents as follows:

$$O\left(|N|U\mathcal{B}g_s\sum_{l=1}^{2} g^a_l g^a_{l+1}\right), \tag{46}$$

where, U is the train epoch, g_s is the number of trains in an epoch, $\mathcal{B}$ denotes the batch size in a train. After training the LRMA, the complexity of the agent when reasoning mainly relies on the Actor module, which can be expressed as $O(|N|Max_n \sum_{l=0}^{2} g^a_l g^a_{l+1})$.

VI. Simulation Experiments

In this section, we evaluate the effectiveness and superiority of the proposed LRMA algorithm by simulating a real-time IoT system based on a real workload trace.

A. Experimental Setup

1) Simulation System: In this study, we set up a simulated real-time IoT system environment that includes N EDs and M MESs. During the experiment, each ED generates tasks with unpredictable types, attributes, and quantities within 300 seconds, reflecting the uncertainty typical in real IoT environments. Additionally, the future states of all EDs and MESs are unknown, which helps simulate dynamic changes in device status and tests the adaptability and robustness of the algorithm against unstable

TABLE II
Main Parameters and Meaning

Parameters name	Value	Parameters name	Value
N	20, 25, 30	M	5
V	20	κ	10^{-24}
$\|R\|$	3	Max_n	5
K	300 seconds	τ	1 second
ω	10^8 bits	σ^2	-60 dBm

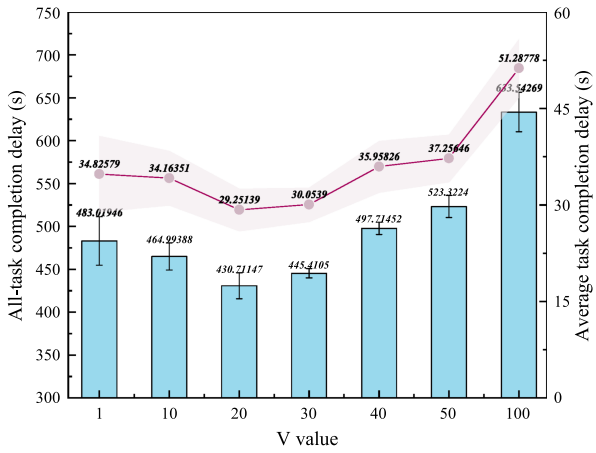

Fig. 4. Impact of different V values on IoT system processing capacity.

factors. Our data are derived from a dataset containing 1523 heterogeneous GPU computing nodes from Alibaba Cloud's cluster [11]. After thorough data cleaning, we obtained 26925 usable heterogeneous GPU production tasks. In the experiments, the system randomly selects tasks from this dataset for the EDs at the start of each time frame, ensuring the real-time and realistic nature of the experiment.

All computations were conducted on servers: an NVIDIA Tesla V100 GPU and an Intel Xeon Silver 4214 CPU. Some experimental parameters are shown in Table II, with the parameter settings for the simulation experiments primarily derived from references [27] and [35].

2) Comparative Experimental Design: To validate the superiority of our proposed MHFQ framework, we compared it with two common MES task processing frameworks:

1) *FCFS Queuing Framework [26], [27]:* The queue adopts a first-come, first-served discipline, processing tasks in the order of their arrival.
- *Advantage:* Simple and intuitive; can respond quickly.
- *Disadvantage:* Unable to process tasks in parallel.

2) *M/M/C Queuing Framework [29], [30]:* The framework allocates MES computational resources to multiple queues to enable parallel processing of large-scale tasks.
- *Advantage:* Accurately model complex IoT system.
- *Disadvantage:* No guarantee of multitasking fairness.

To validate the performance of the LRMA algorithm, we compared it with three task offloading real-time decision-making algorithms:

1) *DVCCO algorithm [35]:* An LSTM-based deep Q-network (DQN) reinforcement learning algorithm. This algorithm uses historical data and task state information to make offloading decisions for each task. Its goal is to maximize the task processing capability of EDs while facing unknown future states.

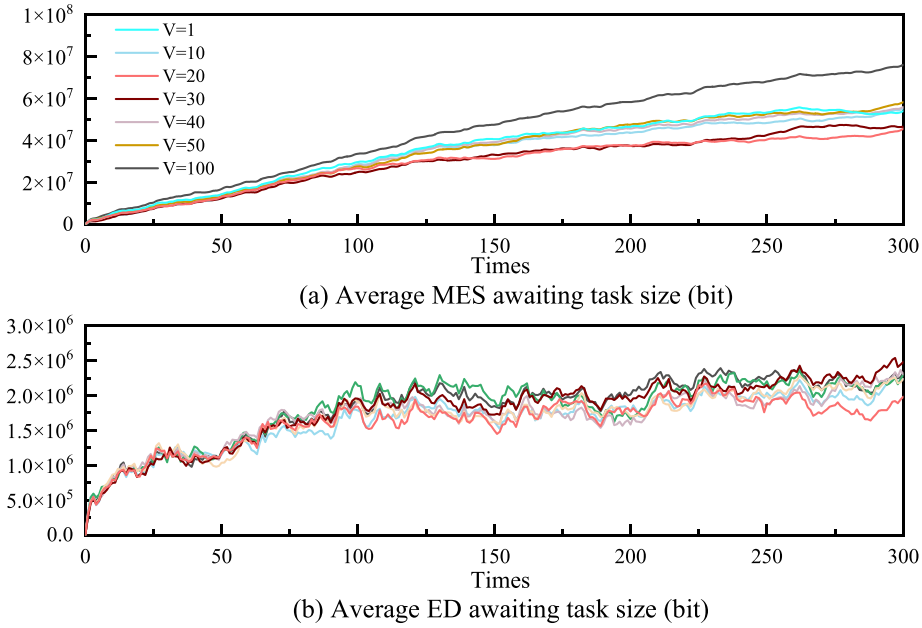

Fig. 5. Impact of different V values on IoT system fluctuations.

2) *MA3MCO algorithm [36]:* A multi-agent DRL algorithm uses two agent networks for each agent, each network responsible for processing decisions for different goals.
3) *L-MADDPG algorithm [37]:* A multi-agent DRL algorithm using Deep Deterministic Policy Gradient (DDPG) networks for efficient resource scheduling and allocation.

To evaluate the performance of our proposed LRMA algorithm and the MHFQ scheme, we conducted simulation experiments covering three aspects: *1) Ablation experiments:* Optimization of the Lyapunov penalty parameter V and assessment of the parameter reset method efficacy; *2) MHFQ framework evaluation:* Validation of the MHFQ effectiveness in IoT scenarios with varying number of tasks; *3) LRMA algorithm assessment:* Validating the LRMA algorithm's performance from different task generation rates.

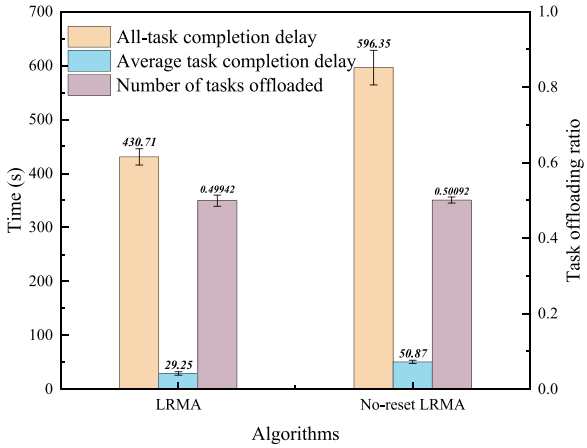

Fig. 6. Effectiveness validation of network parameter reset strategy.

B. Ablation Experiment Analysis

1) Impact of Lyapunov Control Parameter: Fig. 4 illustrates the impact of different V values on the total processing delay and average processing time of tasks generated by the IoT system over 300 seconds. The results indicate that as V increases from 1 to 100, both metrics initially decrease and then increase, with the minimum values occurring at $V = 20$. Fig. 5 shows the effect of different V values on the queue fluctuations in ED and MES. The results reveal that the average task accumulation in the ED and MES queues is lowest at $V = 20$, with minimal fluctuations between adjacent time.

Our analysis of the results in Figs. 4 and 5 shows that when the V value is low, the LRMA algorithm focuses more on maintaining the stability of queues in the IoT system. This stability helps balance the task distribution across servers, thereby enhancing the system's overall processing capacity. Conversely, with higher V values, the algorithm shifts its focus to optimizing task processing, which can result in less effective offloading strategies. Specifically, the LRMA algorithm tends to offload a large number of tasks to the high-capacity MES at each time step, leading to congestion in the MES queue. This congestion increases task waiting times and ultimately raises the average completion time for each task. Therefore, selecting the appropriate V value is crucial for balancing the IoT system's processing capacity and buffering needs. Based on the experimental results, we have chosen $V = 20$ as the Lyapunov parameter for the subsequent experiments.

2) Impact of Parameter Reset Method: Figs. 6 and 7 depict the impact of the parameter reset module on the LRMA algorithm. Fig. 6 illustrates the performance of the LRMA algorithm with periodic parameter resets in terms of All-task processing time, average processing time, and offloading rate. The graph demonstrates that adopting LRMA with periodic network parameter resets effectively reduces task processing time and average processing time by approximately 28% and 33%, respectively. This improvement stems from addressing Primacy Bias during agent learning, where early data overfitting may

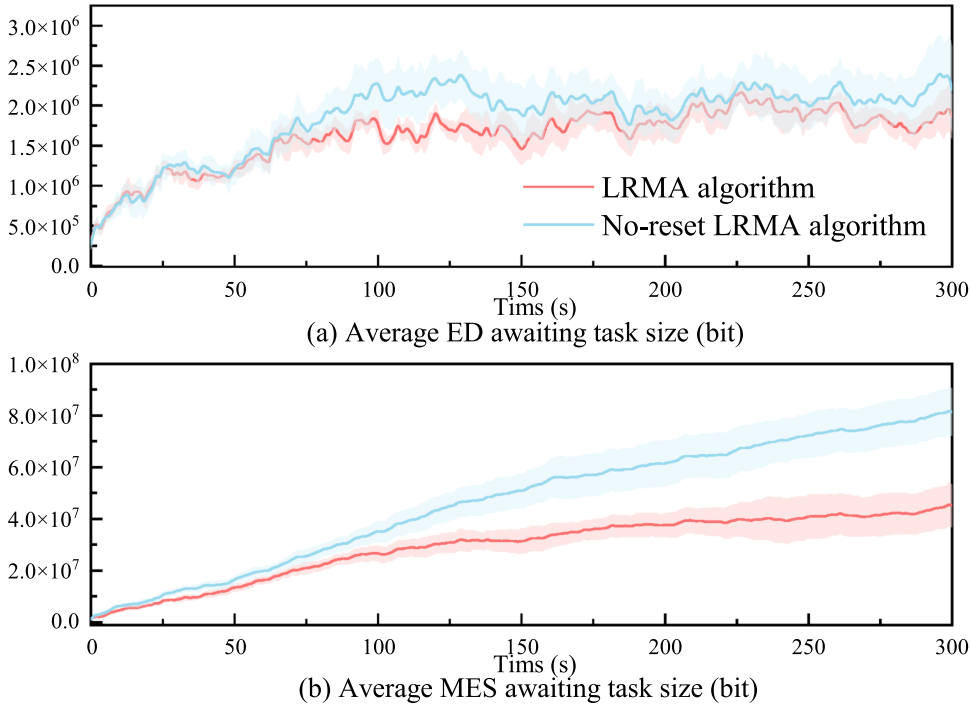

Fig. 7. Effectiveness validation of network parameter reset strategy.

impair future decision-making capabilities. Regularly resetting certain agent parameters mitigates primacy bias [21].

Fig. 7 further highlights the robust queue stability achieved by the LRMA algorithm. This stability is attributed to LRMA's ability to make rational task offloading decisions, thereby offloading more tasks to the cloud. Overall, periodic parameter resets enhance LRMA performance by reducing processing times and ensuring stable task management, crucial for efficient IoT system operation.

C. Performance Assessment

1) MHFQ Framework Evaluation: In Fig. 8, we depict the impact of MES queue frameworks on the task-processing capabilities of IoT systems across different user scales. Fig. 8(a) illustrates the total and average time for complete All-task processing over a 300-second interval. It is evident from the figure that the IoT system utilizing the MHFQ framework outperforms other frameworks across all metrics, achieving reductions of approximately 16.4% in All-task completion time and 21.1% in average processing delay. This advantage arises from the MHFQ framework's enhancement of resource allocation fairness, effectively reducing queue times for micro-tasks and enabling the system to perform well under varying task loads.

Fig. 8(b), (c) and (d) present variations in the average awaiting task size under different user counts. While all queue curve under ED exhibit similar fluctuations, the MES curve employing the MHFQ framework demonstrates significantly reduced pending task sizes, approximately 18.4% and 17.3% lower than those under FCFS and M/M/C queue frameworks, respectively. This outcome further validates the MHFQ framework's adaptive resource allocation capability, facilitating prompt processing of large-scale tasks with low resource demands and mitigating queue blockage risks associated with prolonged resource monopolization. In summary, the MHFQ framework enhances system responsiveness, fairness, and resource utilization efficiency, making it well-suited for addressing the complexities and variability of IoT systems.

2) LRMA Algorithm Performance Evaluation: Fig. 9 demonstrates the impact of different offloading decision algorithms on the processing performance of IoT systems. Firstly, from Fig. 9(a), we can see that under varying task generation rates, the LRMA algorithm consistently reduces All-task completion time significantly, with its performance becoming increasingly superior as task generation rates rise. In Fig. 9(b), the LRMA algorithm achieves a peak task offloading rate close to 50%, showcasing its advantage in utilizing MES computational resources. In contrast, the DVCCO algorithm exhibits very low task offloading rates, negatively correlating with task generation rates. There are three main reasons for this situation: (i) All actions are generated by the same DQN network, leading to competition for information and difficulty in balancing action importance during learning; (ii) The Actor module is deployed on the ED side, relying solely on local observations, which limits its global perspective and decision-making accuracy; (iii) The LSTM module also depends on local states, hindering its ability to predict task arrivals at each MES accurately. These factors negatively impact the DVCCO algorithm's performance, reducing its adaptability and accuracy in complex dynamic environments.

Fig. 10 shows the variations in task sizes within ED and MES virtual queues under different algorithms and task generation rates. The ED task queue size is lowest with the LRMA algorithm, while the MES queue size is highest, indicating LRMA's effectiveness to offload most tasks to the MES. At lower task generation rates, all algorithms exhibit significant fluctuations in queue sizes due to sufficient IoT system resources. However, as task generation rates increase, these fluctuations decrease

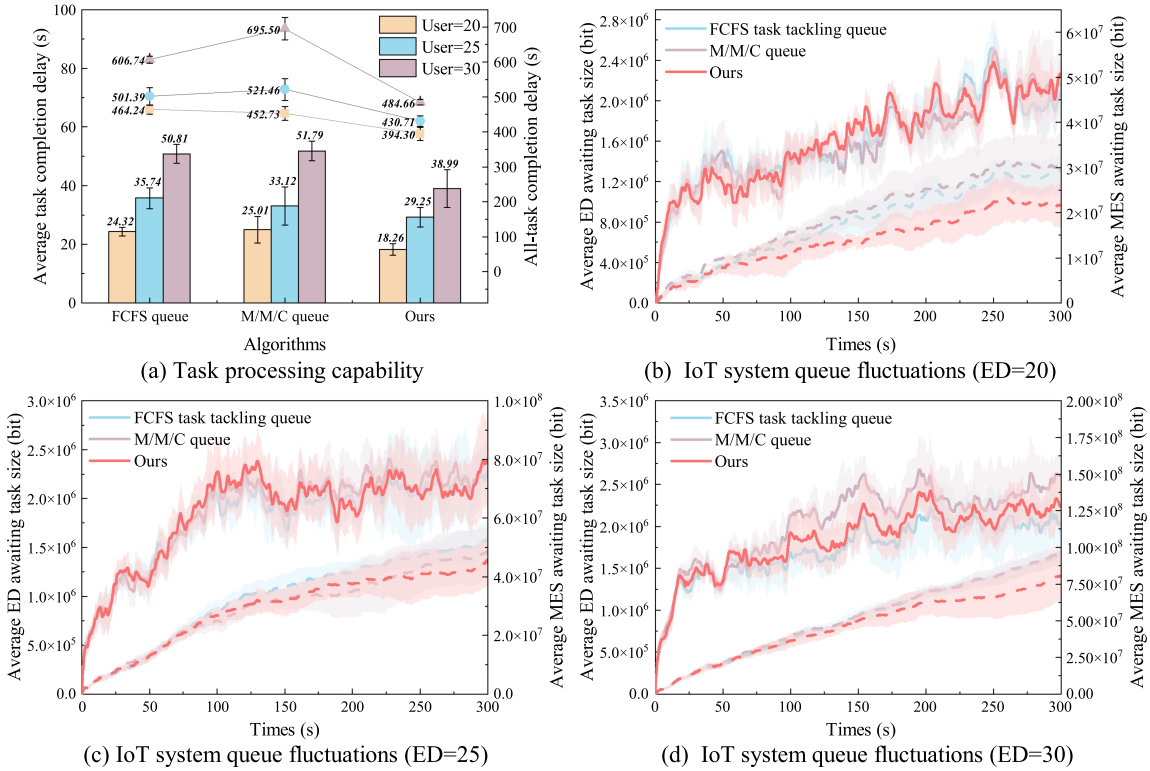

Fig. 8. Comparison of MHFQ framework performance.

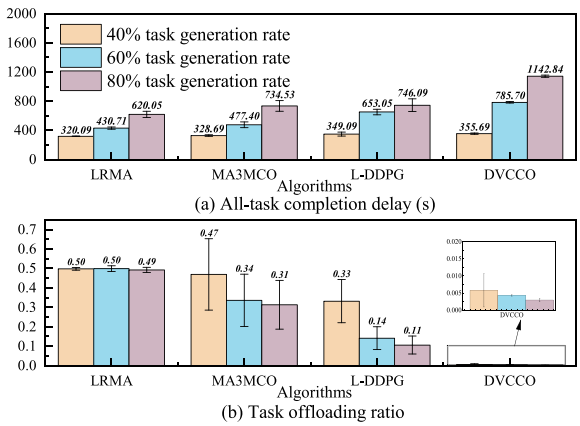

Fig. 9. Impact of different algorithms on IoT processing power.

significantly. Compared to others, LRMA shows superior performance and lower error rate. This is because IoT systems face computational resource shortages as the number of tasks increases, necessitating efficient decision-making solutions. However, all three comparative algorithms are influenced by the limited observability of ED and the constrained resources available for their execution. Among these algorithms, the DVCCO algorithm exhibits the poorest performance. This is primarily due to its LSTM module, which predicts a continued scarcity of system resources, which further guides the algorithm for local task processing

In summary, Figs. 9 and 10 validate the advantages of the LRMA algorithm in optimizing IoT system processing capabilities, enhancing MES computational resource utilization, and reducing task queue fluctuations. Moreover, the adaptability of LRMA in task-intensive large-scale hybrid computing task scenarios is demonstrated.

VII. Conclusion

This study investigates the problem of online offloading of large-scale heterogeneous tasks in dynamic IoT environments. We formulate the original problem as a multi-stage MINLP problem and decouple it into a single-stage deterministic MINLP problem using Lyapunov optimization. Building upon this formulation, we propose a multi-agent reinforcement learning algorithm that combines LSTM and parameter resetting methods. This algorithm predicts states and generates offloading decisions for each task in near real-time, addressing uncertainties in future IoT system environments. Additionally, we introduce a MHFQ task processing framework based on three-level virtual queues to achieve parallel processing of large-scale tasks. Finally, We conduct extensive experiments to validate the effectiveness and superiority of the proposed algorithms and frameworks.

Based on our current research, we plan to further explore the scheduling and resource allocation of heterogeneous computing tasks in dynamic IoT systems. Our future work will focus on two main aspects: (i) investigating more complex scheduling strategies, such as the allocation of time slices and adjustment of task priorities that better match the real server operation; and (ii) addressing issues related to the fragmentation of resources such as CPU and memory, as well as the suboptimal placement of different services.

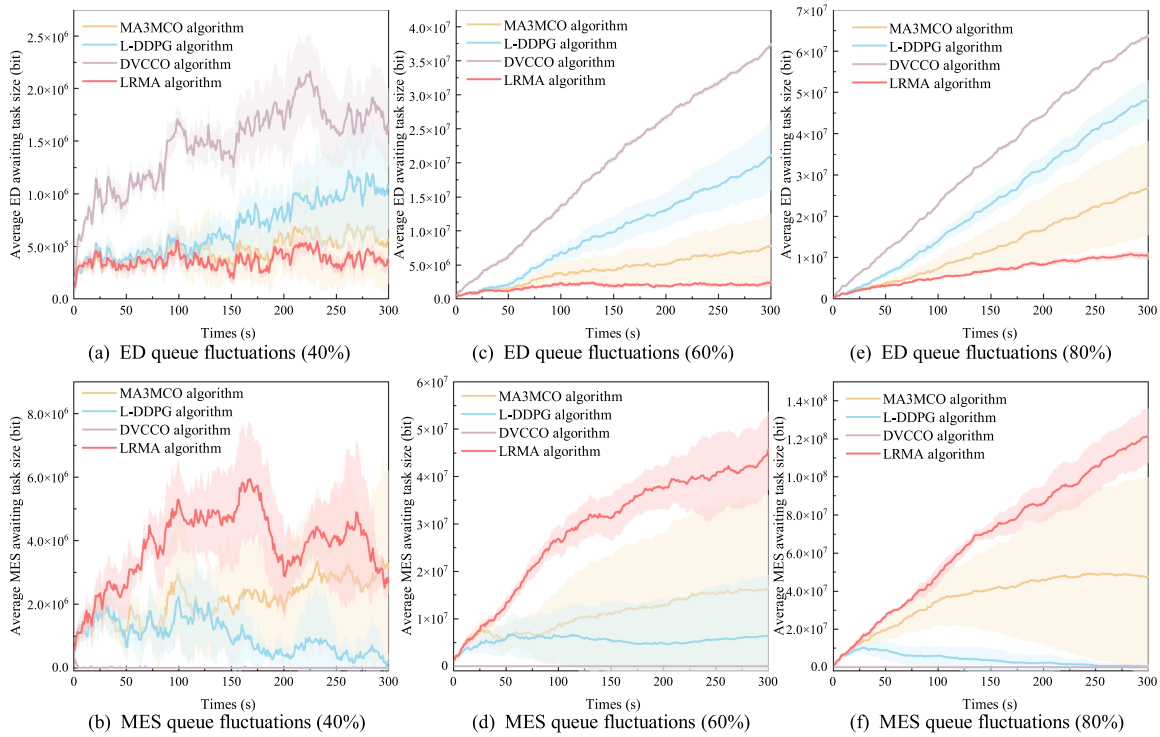

Fig. 10. Comparison of IoT awaiting task curve fluctuations based on different algorithms. (a) and (b) depict the average awaiting task size and its variability for ED and MES under a task arrival probability of 40%. (c) and (d) illustrate the comparisons at task arrival probability of 60%. (e) and (f) represent the comparisons at task arrival probability of 80%.

REFERENCES

[1] S. Xia, Z. Yao, and Y. Li, "A distributed stochastic task offloading methodology for IoT on e-health," in *Proc. 2020 IEEE Int. Conf. Commun.*, 2020, pp. 1–6.

[2] W. Z. Khan, E. Ahmed, S. Hakak, I. Yaqoob, and A. Ahmed, "Edge computing: A survey," *Future Gener. Comput. Syst.*, vol. 97, pp. 219–235, 2019.

[3] Z. Ning et al., "Intelligent edge computing in Internet of Vehicles: A joint computation offloading and caching solution," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2212–2225, Apr. 2021.

[4] Z. Gao, L. Yang, and Y. Dai, "Large-scale computation offloading using a multi-agent reinforcement learning in heterogeneous multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3425–3443, Jun. 2023.

[5] R. NoorianTalouki, M. H. Shirvani, and H. Motameni, "A heuristic-based task scheduling algorithm for scientific workflows in heterogeneous cloud computing platforms," *J. King Saud Univ.- Comput. Inf. Sci.*, vol. 34, no. 8, pp. 4902–4913, 2022.

[6] S. Xia, Z. Yao, Y. Li, and S. Mao, "Online distributed offloading and computing resource management with energy harvesting for heterogeneous MEC-enabled IoT," *IEEE Trans. Wireless Commun.*, vol. 20, no. 10, pp. 6743–6757, Oct. 2021.

[7] N.-M. Ho and W.-F. Wong, "Tensorox: Accelerating GPU applications via neural approximation on unused tensor cores," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 2, pp. 429–443, Feb. 2021.

[8] C. Gong, M. Ma, L. Wu, W. Liu, Y. Zhou, and Y. Yang, "Task offloading and resource allocation in CPU-GPU heterogeneous networks," in *Proc. 2022 IEEE Glob. Commun. Conf.*, 2022, pp. 4492–4497.

[9] A. Dhakal, S. G. Kulkarni, and K. Ramakrishnan, "Machine learning at the edge: Efficient utilization of limited CPU/GPU resources by multiplexing," in *Proc. 2020 IEEE 28th Int. Conf. Netw. Protoc.*, 2020, pp. 1–6.

[10] N. Tsog, S. Mubeen, F. Bruhn, M. Behnam, and M. Sjödin, "Offloading accelerator-intensive workloads in CPU-GPU heterogeneous processors," in *Proc. 2021 26th IEEE Int. Conf. Emerg. Technol. Factory Automat.*, 2021, pp. 1–8.

[11] Q. Weng et al., "Beware of fragmentation: Scheduling {GPU-Sharing} workloads with fragmentation gradient descent," in *Proc. 2023 USENIX Annu. Techn. Conf.*, 2023, vol. 23, pp. 995–1008.

[12] M. Kumar, A. Kishor, J. K. Samariya, and A. Y. Zomaya, "An autonomic workload prediction and resource allocation framework for fog enabled industrial IoT," *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9513–9522, Jun. 2023.

[13] G. Ma, X. Wang, M. Hu, W. Ouyang, X. Chen, and Y. Li, "DRL-based computation offloading with queue stability for vehicular-cloud-assisted mobile edge computing systems," *IEEE Trans. Intell. Veh.*, vol. 8, no. 4, pp. 2797–2809, Apr. 2023.

[14] G. K. Walia, M. Kumar, and S. S. Gill, "AI-empowered FOG/EDGE resource management for IoT applications: A comprehensive review, research challenges and future perspectives," *IEEE Commun. Surveys Tut.*, vol. 26, no. 1, pp. 619–669, Firstquarter 2024.

[15] F. Chai, Q. Zhang, H. Yao, X. Xin, R. Gao, and M. Guizani, "Joint multi-task offloading and resource allocation for mobile edge computing systems in satellite IoT," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 7783–7795, Jun. 2023.

[16] M. Kumar, A. Kishor, J. Abawajy, P. Agarwal, A. Singh, and A. Y. Zomaya, "ARPS: An autonomic resource provisioning and scheduling framework for cloud platforms," *IEEE Trans. Sustain. Comput.*, vol. 7, no. 2, pp. 386–399, Apr.–Jun. 2022.

[17] K. Hwang and N. Jotwani, *Advanced Computer Architecture: Parallelism, Scalability, Programmability*. New York, NY, USA: McGraw-Hill, 1993, vol. 199.

[18] M. Kumar, G. K. Walia, H. Shingare, S. Singh, and S. S. Gill, "AI-based sustainable and intelligent offloading framework for IIoT in collaborative cloud-fog environments," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 1414–1422, Feb. 2024.

[19] E. Wang, D. Li, B. Dong, H. Zhou, and M. Zhu, "Flat and hierarchical system deployment for edge computing systems," *Future Gener. Comput. Syst.*, vol. 105, pp. 308–317, 2020.

[20] S. Long, W. Long, Z. Li, K. Li, Y. Xia, and Z. Tang, "A game-based approach for cost-aware task assignment with QoS constraint in collaborative edge and cloud environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1629–1640, Jun. 2021.

[21] E. Nikishin, M. Schwarzer, P. D'Oro, P.-L. Bacon, and A. Courville, "The primacy bias in deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 16828–16847.
[22] M. Kumar, A. Kishor, P. K. Singh, and K. Dubey, "Deadline-aware cost and energy efficient offloading in mobile edge computing," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 5, pp. 778–789, Sep./Oct. 2024.
[23] X. Tang, W. Shi, and F. Wu, "Interconnection network energy-aware workflow scheduling algorithm on heterogeneous systems," *IEEE Trans. Ind. Informat.*, vol. 16, no. 12, pp. 7637–7645, Dec. 2020.
[24] Z. Zhao, H. Zhang, L. Wang, and H. Huang, "A multi-model edge computing offloading framework for deep learning application based on Bayesian optimization," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 18387–18399, Oct. 2023.
[25] H. Jiang, X. Dai, Z. Xiao, and A. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4000–4015, Jul. 2023.
[26] M. Maray, E. Mustafa, J. Shuja, and M. Bilal, "Dependent task offloading with deadline-aware scheduling in mobile edge networks," *Internet Things*, vol. 23, 2023, Art. no. 100868.
[27] S. Bi, L. Huang, H. Wang, and Y.-J. A. Zhang, "Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 11, pp. 7519–7537, Nov. 2021.
[28] S. Bebortta, D. Senapati, C. R. Panigrahi, and B. Pati, "Adaptive performance modeling framework for QoS-aware offloading in MEC-based IIoT systems," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 10162–10171, Jun. 2022.
[29] S. Guan and A. Boukerche, "A novel mobility-aware offloading management scheme in sustainable multi-access edge computing," *IEEE Trans. Sustain. Comput.*, vol. 7, no. 1, pp. 1–13, Jan.–Mar. 2022.
[30] E. Hosseini, M. Nickray, and S. Ghanbari, "Energy-efficient scheduling based on task prioritization in mobile fog computing," *Computing*, vol. 105, no. 1, pp. 187–215, 2023.
[31] I. A. Qazi and T. Znati, "On the design of load factor based congestion control protocols for next-generation networks," *Comput. Netw.*, vol. 55, no. 1, pp. 45–60, 2011.
[32] Y. Zhao, B. Li, J. Wang, D. Jiang, and D. Li, "Integrating deep reinforcement learning with pointer networks for service request scheduling in edge computing," *Knowl.-Based Syst.*, vol. 258, 2022, Art. no. 109983.
[33] H. Wang, M. Sun, L. Zhang, P. Dong, Y. Wei, and J. Mei, "Scheduling optimization for upstream dataflows in edge computing," *Digit. Commun. Netw.*, vol. 9, no. 6, pp. 1448–1457, 2023.
[34] F. D. Tilahun, A. T. Abebe, and C. G. Kang, "Multi-agent reinforcement learning for distributed resource allocation in cell-free massive MIMO-enabled mobile edge computing network," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16454–16468, Dec. 2023.
[35] G. Ma, X. Wang, M. Hu, W. Ouyang, X. Chen, and Y. Li, "DRL-based computation offloading with queue stability for vehicular-cloud-assisted mobile edge computing systems," *IEEE Trans. Intell. Veh.*, vol. 8, no. 4, pp. 2797–2809, Apr. 2023.
[36] J. Cai, H. Fu, and Y. Liu, "Multitask multiobjective deep reinforcement learning-based computation offloading method for industrial Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 2, pp. 1848–1859, Jan. 2023.
[37] A. S. Kumar, L. Zhao, and X. Fernando, "Task offloading and resource allocation in vehicular networks: A Lyapunov-based deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 10, pp. 13360–13373, Oct. 2023.

Xiao He received the master's degree fromthe China University of Petroleum, Beijing, China, in 2023, where he is currently working toward the Ph.D. degree. His research interests include Industry Internet, edge computing, and deep learning.

Shanchen Pang received the graduation degree from the Tongji University of Computer Software and Theory, Shanghai, China, in 2008. He is currently a Professor from the China University of Petroleum, Qingdao, China. His research interests include theory and application of Petri Net, Service computing, Trusted computing, and Artificial intelligence.

Haiyuan Gui received the B.S degree from Qingdao University of Technology, Qingdao, China, in 2020. He is currently working toward the Ph.D. degree with the College of Computer Science and Technology from China University of Petroleum (East China), Qingdao, China. His research interests include reinforcement learning, meta learning, and transfer learning.

Kuijie Zhang is currently working toward the Ph.D. degree with the China University of Petroleum (East China), Qingdao, China. His research interests include artificial intelligence, and data mining.

Nuanlai Wang (Graduate Student Member, IEEE) received the master's degree from the China University of Petroleum (East China), Qingdao, China, in 2022, where he is currently working toward the Ph.D. degree. His research interests include intelligent transportation system, edge computing, and internet of vehicles.

Xue Zhai received the B.S. degree from Qingdao University of Technology, Qingdao, China, in 2018. She is a graduate student of China University of Petroleum(East China), Qingdao. Her research interest is End-edge-cloud computing and blockchain.